

# Resumo da Disciplina de Segurança Informática (por Temas)

---

Este documento consolida, por tema e progressão lógica, os conteúdos das aulas lecionadas (Aula01 a Aula20). A disciplina está organizada em duas grandes partes: **Mecanismos e Protocolos Criptográficos e Autenticação e Autorização**, culminando na aplicação prática com a biblioteca Casbin.

---

## 1. Fundamentos de Segurança e Criptografia

### 1.1 Propriedades Básicas da Segurança da Informação

- **Confidencialidade**: impedir a divulgação não autorizada de informação. Ex.: extrato bancário acessível apenas pelo banco e pelo cliente.
- **Integridade**: garantir que a informação não sofreu alterações não autorizadas (adição, remoção, reordenação). Inclui a **autenticidade** (origem fidedigna), que por sua vez permite o **não-repúdio** (o autor não pode negar a criação).
- **Disponibilidade**: o sistema/dados devem estar acessíveis sempre que necessário. Ataques típicos: DoS/DDoS.
- **Privacidade**  $\neq$  confidencialidade: privacidade define *quem* pode aceder a quais dados pessoais.

### 1.2 Ataques Clássicos

- **Divulgação de conteúdo** (*information disclosure*): ataque passivo; observação da informação (ex.: *sniffing*).
- **Análise de tráfego**: passivo; inferência por padrões de comunicação.
- **Disfarce** (*spoofing*): ativo; atacante faz-se passar por entidade legítima.
- **Repassé / Repetição** (*replay*): ativo; reenvio de mensagens capturadas.
- **Man-In-The-Middle**: interposição ativa, interceptação e modificação.
- **Negação de serviço** (DoS/DDoS): sobrecarga para indisponibilizar o serviço.

### 1.3 Introdução à Criptografia

- **Criptografia**: ciência de escrever mensagens cifradas; **criptoanálise**: quebra sem a chave; **criptologia**: ambos.
- Elementos fundamentais:
  - Texto plano ( $m$ ), texto cifrado ( $c$ ).
  - Função de cifra ( $E(.)$ ) e de decifra ( $D(.)$ ), parametrizadas por **chave criptográfica** ( $k$ ).
  - ( $c = E(k)(m)$ ), ( $m' = D(k)(c)$ ); correção exige ( $D(k)(E(k)(m)) = m$ ).
- **Cifra de César**: substituição monoalfabética com deslocamento fixo ( $k$ ) (chave).
  - ( $E(k)(m_i) = (m_i + k) \bmod 26$ ). Apenas 26 chaves possíveis  $\rightarrow$  vulnerável a **força bruta**.
- **Cifra de Substituição Monoalfabética**: mapeamento arbitrário (permutação do alfabeto). (26!) chaves possíveis, mas quebrável por **análise de frequência** (redundância da língua).
- **Cifra de Vigenère**: polialfabética; usa chave repetida (ex.: "FRUTA") e *tabula recta*.
  - ( $E(k)(m_i) = (m_i + k_{i \bmod |k|}) \bmod 26$ ). Esconde frequências de letra única, mas ainda vulnerável a análises estatísticas mais sofisticadas.

- **Máquina Enigma**: dispositivo eletromecânico com rotores (chave = posição inicial). Criptoanálise explorou padrões (ex.: início de mensagens) e levou à criação da **Bombe** (Turing).
- 

## 2. Hierarquia de Métodos Criptográficos e Primitivas Simétricas

### 2.1 Níveis de Abstração

- **Primitiva criptográfica**: algoritmo básico (ex.: AES, DES) com limitações (ex.: opera sobre blocos de tamanho fixo).
- **Esquema criptográfico**: primitiva + métodos auxiliares (modo de operação, *padding*) para uma tarefa específica (confidencialidade, integridade). Ex.: **DES-CBC-PKCS5Padding**.
- **Protocolo criptográfico**: combina esquemas para um objetivo de comunicação seguro (ex.: TLS).

### 2.2 Criptografia Simétrica vs. Assimétrica

- **Simétrica**: mesma chave para cifra e decifra. Rápida, adequada a grandes volumes. Problema: estabelecimento seguro da chave.
- **Assimétrica** (chave pública): par de chaves (pública e privada). Chave pública pode ser divulgada.

### 2.3 Cifras Simétricas e Primitivas de Bloco

- **Cifra simétrica**: (G) gera chave;  $E(k)(m) = c$ ;  $D(k)(c) = m'$ . Deve suportar  $m \in \{0,1\}^*$ .
  - **Propriedades**:
    - **Correção**:  $\forall m, k : D(k)(E(k)(m)) = m$ .
    - **Segurança**: obter (m) a partir de (c) sem (k) é computacionalmente infazível.
    - **Não garante integridade** nem autenticidade por si só (ex.: atacante pode alterar bits do criptograma sem ser detectado).
  - **Primitivas de bloco**: operam sobre blocos de tamanho fixo (ex.: AES 128 bits, DES 64 bits). Tamanho de chave (ex.: AES-128/192/256, DES 56 bits) e bloco influenciam segurança.
  - **DES**: 16 rondas, expansão, substituição. Hoje obsoleto (chave curta).
  - **AES**: *standard* NIST atual, seguro. Derivado do Rijndael.
  - **Cifras de fluxo** (ex.: RC4, modo CTR) operam bit a bit, sem necessidade de padding.
  - Boas práticas: usar primitivas **publicamente conhecidas e padronizadas**, evitar segurança por obscuridade, usar implementações consolidadas.
- 

## 3. Modos de Operação

- **Objetivo**: usar cifras de bloco para mensagens de tamanho arbitrário, garantindo propriedades desejáveis: esconder padrões, eficiência, tamanho, acesso aleatório, correção de erros.

### 3.1 ECB (*Electronic Codebook*)

- **Funcionamento**: cada bloco é cifrado independentemente.  $c_i = E(k)(m_i)$ .
- **Vantagens**: paralelizável, acesso aleatório, erros não se propagam.
- **Desvantagem crítica**: blocos idênticos geram criptogramas idênticos, revelando padrões (ex.: imagem cifrada ainda mostra contornos).

### 3.2 CBC (*Cipher Block Chaining*)

- **Funcionamento:**  $(c_i = E(k)(m_i \oplus c_{i-1}))$ , com  $(c_0 = IV)$  (**vetor de inicialização**).
- **Decifra:**  $(m_i = D(k)(c_i \oplus c_{i-1}))$ .
- **Propriedades:**
  - IV deve ser **imprevisível e não se repetir** (sob a mesma chave). Pode ser enviado em claro.
  - Blocos iguais de texto plano geram criptogramas diferentes (dependendo do contexto).
  - **Propagação de erros:** um bit errado em  $(c_j)$  afeta todo o bloco  $(m_j)$  e os bits correspondentes de  $(m_{j+1})$ .
  - Enfraquece acesso aleatório para escrita e paralelismo.

### 3.3 Padding PKCS#5

- **Necessário** para modos que exigem blocos completos.
- **Funcionamento:** adiciona  $(X)$  bytes (onde  $(X)$  é o número de bytes faltantes) com valor  $(X)$ . Se o tamanho é múltiplo do bloco, adiciona um bloco inteiro de padding.
- **Ataque *padding oracle*:** explorado quando o receptor sinaliza *padding* válido/inválido, permitindo decifrar texto plano sem a chave (Vaudenay 2002).

### 3.4 Modos de Operação em *Stream* (CTR, OFB, CFB)

- Geram um **fluxo de chave**  $(ks_i)$  e fazem XOR com o texto plano.  $(c_i = m_i \oplus ks_i)$ .
- **Não necessitam de padding.**
- **CTR (Counter):** estado  $(l_0 = IV)$ ,  $(l_i = l_{i-1} + 1)$ .  $(ks_i = E(k)(l_i))$ .
  - Acesso aleatório eficiente, paralelizável, erros não se propagam.
  - Segurança depende de IV único por mensagem e chave.

## 4. Esquemas MAC e Cifra Autenticada

### 4.1 Message Authentication Code (MAC)

- **Propósito:** garantir integridade e autenticidade usando chave simétrica compartilhada.
- **Componentes:**
  - $(G)$ : gera chave simétrica  $(k)$ .
  - $(T(k)(m) = t)$  (marca).
  - $(V(k)(m, t) \in \{\text{true}, \text{false}\})$ .
- **Propriedades:**
  - **Correção:**  $(V(k)(m, T(k)(m)) = \text{true})$ .
  - **Segurança:** resistência a falsificação seletiva e existencial (computacionalmente infazível sem  $(k)$ ).
  - Tamanho da marca idealmente fixo (usam-se *hashes* criptográficos).
- **Não garante não-repúdio** (chave compartilhada por múltiplas entidades).
- **Abordagens para construção:** cifrar metadados da mensagem? Não resolve se alterações não alteram metadados. Melhor: usar a própria mensagem como entrada da função.

### 4.2 Cifra Autenticada

- Combina confidencialidade + integridade.
- **Encrypt-then-MAC:**  $(E(k_1)(m) \parallel T(k_2)(E(k_1)(m)))$  (recomendado, verifica autenticidade antes de decifrar).
- **MAC-then-encrypt:**  $(E(k_1)(m \parallel T(k_2)(m)))$  (usado no TLS até 1.2).

- Importante: usar chaves diferentes para cada propósito.
- **Galois Counter Mode (GCM)**: combina CTR com a função GHASH; provê cifra autenticada e permite AAD (*Additional Authentication Data*). Ex.: TLS 1.3 usa AES-GCM.

## 5. Funções Hash Criptográficas

- **Definição**:  $H: \{0,1\}^* \rightarrow \{0,1\}^n$  (ex.: SHA-256,  $n=256$  bits).
- **Propriedades de segurança**:
  - **Resistência à pré-imagem**: dado  $y$ , infazível encontrar  $x$  tal que  $H(x)=y$ .
  - **Resistência à segunda pré-imagem**: dados  $x_1$  e  $H(x_1)$ , infazível encontrar  $x_2 \neq x_1$  com mesmo hash.
  - **Resistência a colisão**: infazível encontrar qualquer par  $x_1 \neq x_2$  com  $H(x_1)=H(x_2)$ . (Implica resistência à segunda pré-imagem.)
- **Não-invertibilidade**: consequência do domínio infinito e contradomínio finito.
- **Aplicações**: verificação de integridade; esquemas MAC (ex.: HMAC); assinaturas digitais; derivação de chaves a partir de palavras-passe; *proof-of-work* (blockchain).

### 5.1 HMAC

- Construção padronizada de MAC baseada em hash:  $HMAC(k, m) = H((k \oplus opad) \parallel H((k \oplus ipad) \parallel m)))$ .
- ( $opad = 0x5c5c\dots5c$ ), ( $ipad = 0x3636\dots36$ ).
- Exemplo: **HMAC-SHA256**. Tamanho da marca fixo para a função hash escolhida.

## 6. Criptografia Assimétrica e Assinaturas Digitais

### 6.1 Esquemas Assimétricos

- **Chave pública** ( $k_{\text{pub}}$ ) (pode ser divulgada) e **chave privada** ( $k_{\text{priv}}$ ) (secreta).
- Propriedade:  $(D(k_{\text{priv}})(E(k_{\text{pub}})(m))) = m$  e vice-versa (para RSA).
- **Cifra assimétrica**: mensagem cifrada com chave pública do receptor; decifrada com chave privada.
- **Assinatura digital**: mensagem "assinada" (geração de marca) com chave privada; verificada com chave pública.

### 6.2 RSA (Rivest–Shamir–Adleman)

- **Geração de chaves**:
  1. Escolher dois primos aleatórios grandes ( $P, Q$ ).
  2.  $(N = P \cdot Q)$ .
  3. Calcular  $(\lambda(N) = \text{lcm}(P-1, Q-1))$ .
  4. Escolher  $(E)$  (expoente público) e  $(D)$  (expoente privado) tais que  $(E \cdot D \equiv 1 \pmod{\lambda(N)})$ .
    - Chave pública:  $((E, N))$ ; chave privada:  $((D, N))$ .
- **Cifra**:  $(C = M^E \bmod N)$  (com  $(M < N)$ ).
- **Decifra**:  $(M = C^D \bmod N)$ .
- A segurança baseia-se na dificuldade de fatorar  $(N)$  (problema do logaritmo discreto em computadores quânticos pode ameaçar no futuro).

- **Limitação:** mensagens devem ser codificadas como números menores que (N); usam-se métodos de formatação (*padding* como OAEP) para evitar casos especiais (ex.: (M=0)).
- **Uso típico:** esquema híbrido – RSA cifra chave de sessão simétrica, que depois cifra os dados.

## 6.3 Assinatura Digital

- **Processo:**
    1. Calcular ( $h = H(m)$ ) (hash da mensagem).
    2. Assinar: ( $s = S(k_{\text{priv}})(m) = E(k_{\text{priv}})(h)$ ).
    3. Verificar: decifrar com chave pública para obter ( $h' = D(k_{\text{pub}})(s)$ ); comparar com ( $H(m)$ ).
  - **Componentes:** função hash, método de formatação (ex.: PKCS#1 v1.5, PSS) e primitiva assimétrica (RSA, DSA).
  - **Propriedade:** garante não-repúdio (apenas detentor da chave privada pode gerar assinatura).
  - **Custo computacional** elevado comparado a MAC (usa exponenciação modular).
- 

## 7. Java Cryptography Architecture (JCA)

### 7.1 Conceitos Fundamentais

- **Engine Classes:** interfaces abstratas para tarefas criptográficas (`Cipher`, `Mac`, `Signature`, `MessageDigest`, `KeyGenerator`, `KeyPairGenerator`, `SecureRandom`). Instanciadas via `getInstance("algoritmo")` (padrão *Factory*).
- **Independência de algoritmo:** código genérico, fácil alterar algoritmo (ex.: "SHA-256" para "MD5").
- **Independência de implementação:** *providers* trocáveis (Sun, SunJCE, etc.) com ordem de prioridade configurável.
- **Specification Classes:** representações transparentes de parâmetros (`SecretKeySpec`, `IvParameterSpec`, `RSAPublicKeySpec`).

### 7.2 Operações Básicas

- **Cifra:**
  - Inicialização: `cipher.init(modo (ENCRYPT_MODE/DECRYPT_MODE), chave, [iv])`.
  - Uso: `byte[] resultado = cipher.doFinal(dados)`. Para fluxos: `update()` + `doFinal()`, ou usar `CipherInputStream/CipherOutputStream`.
- **MAC:** `Mac.getInstance("HmacSHA256"), init(key), doFinal(dados)`.
- **Assinatura:** `Signature.getInstance("SHA256withRSA"), initSign(privKey), update(dados), sign()`; verificação com `initVerify(pubKey)` e `verify(assinatura)`.
- **Digest (hash):** `MessageDigest.getInstance("SHA-256"), digest(dados)`.
- **Geração de chaves:**
  - Simétrica: `KeyGenerator.getInstance("AES"), init(tamanho), generateKey()`.
  - Assimétrica: `KeyPairGenerator.getInstance("RSA"), initialize(2048), generateKeyPair()`.
- **Parâmetros:** `AlgorithmParameters` (opaco), `AlgorithmParameterSpec` (transparente, ex.: `IvParameterSpec`).
- **KeyStore:** armazenamento seguro de chaves privadas e certificados. Formatos: JKS, JCEKS, PKCS12. Carregado com `load(stream, password)`.

## 7.3 Certificados Java Certification Path API

- **Classes:** `CertificateFactory` (gerar objetos `X509Certificate` a partir de streams), `CertPath` (caminho de certificação), `PKIXParameters` (trust anchors e configurações), `CertPathValidator` (validar), `CertPathBuilder` (construir caminho automaticamente), `TrustAnchor`, `CertStore`.
  - **Validação:** verifica assinaturas, datas, confiança até um *trust anchor*, e opcionalmente revogação (CRLs).
  - **Exemplo:** criar `CertPath` com certificados folha e intermédios, carregar `TrustAnchor` (raiz), validar com `CertPathValidator.getInstance("PKIX").validate(certPath, params)`.
- 

## 8. Infraestrutura de Chave Pública (PKI) e Certificados X.509

### 8.1 Necessidade do Certificado Digital

- A distribuição da chave pública está sujeita a **Man-In-The-Middle**: atacante substitui a chave legítima pela sua.
- Solução: uma **Autoridade de Certificação (CA)** emite um **certificado digital** que associa a identidade da entidade à sua chave pública, assinado digitalmente pela CA.
- **Certificado raiz:** auto-assinado, confiável implicitamente (*trust anchor*). Instalado previamente no sistema.

### 8.2 Padrão X.509

- Estrutura definida em ASN.1, codificada em DER.
- **Campos principais:** versão, número de série, emissor (*issuer*), sujeito (*subject*), validade (datas), chave pública, assinatura.
- **Extensões** (perfil PKIX):
  - **Key Usage:** usos permitidos (ex.: assinar certificados).
  - **Basic Constraints:** indica se é CA; limita profundidade do caminho de certificação.
  - **Subject Alternative Name:** nomes DNS alternativos (ex.: `*.youtube.com`).
  - **CRL Distribution Points:** localização da lista de certificados revogados.

### 8.3 Validação de Cadeia

- Processo recursivo: verifica assinatura de cada certificado até encontrar um certificado raiz confiável e não expirado.
  - **Revogação** consultando a CRL. Um certificado pode ser revogado se a chave privada for comprometida.
- 

## 9. Transport Layer Security (TLS)

### 9.1 Objetivos e Arquitetura

- Prover canal seguro (confidencialidade, integridade, autenticação) sobre TCP.
- Dois sub-protocolos:
  - **Record Protocol:** fragmenta, opcionalmente comprime, e aplica cifra/MAC aos dados. Cada registo tem *Content-Type*, *Length*, número de sequência (não transmitido, usado no MAC).
  - **Handshake Protocol:** negociação de parâmetros, autenticação, estabelecimento de chaves.

## 9.2 Handshake (Exemplo RSA, TLS 1.2)

1. **ClientHello**: versão TLS, lista de *cipher suites*, *client random*.
  2. **ServerHello**: *cipher suite* escolhida, *server random*.
  3. **Certificate**: servidor envia seu certificado (corrente X.509).
  4. **ServerHelloDone**.
  5. **ClientKeyExchange**: cliente gera *pre\_master\_secret* e o envia cifrado com a chave pública do servidor.
  6. Ambos derivam *master\_secret* = PRF(*pre\_master\_secret*, "master secret", *client\_random*+*server\_random*).
  7. A partir do *master\_secret*, geram chaves de sessão (cifra, MAC, IV) para cada direção.
  8. Cliente envia *ChangeCipherSpec*, depois *Finished* (HMAC de todas as mensagens de handshake).
  9. Servidor responde com *ChangeCipherSpec* e *Finished*.
- **Autenticação do servidor**: provada pela capacidade de decifrar o *pre\_master\_secret*.
  - **Autenticação do cliente** (opcional): *CertificateRequest* e *CertificateVerify* (assinatura sobre handshake).

## 9.3 Cipher Suites e Perfect Forward Secrecy

- Exemplo: *TLS\_RSA\_WITH\_AES\_128\_GCM\_SHA256* (RSA para key exchange, AES-128-GCM para cifra, SHA256 para MAC).
- **Problema**: se a chave privada do servidor for comprometida, todas as comunicações passadas baseadas em RSA podem ser decifradas (não oferece **Perfect Forward Secrecy – PFS**).
- **Diffie-Hellman (DH) efêmero** (DHE, ECDHE) garante PFS:  $\text{pre\_master\_secret} = g^{ab} \bmod p$  nunca é transmitido, é efêmero. TLS 1.3 exige apenas *cipher suites* com PFS.

## 9.4 TLS em Java

- **Classes**: *SSLContext* (configuração), *SSLSocketFactory*, *SSLSocket*, *SSLSession*.
- **Configuração de confiança**: *TrustManager* (carrega *trust anchors* de uma *KeyStore*).
- **Exemplo**: criar *SSLContext*, inicializar com *TrustManager*, obter *SSLSocket*, *startHandshake()*, obter streams.

## 9.5 HTTPS e Ataques a CAs

- HTTPS = HTTP sobre TLS (porta 443). O certificado deve conter o *hostname* no *Common Name* ou *SubjectAltName*.
- **Ataque à DigiNotar (2011)**: emissão de certificados falsos para Google, usados em MITM.

# 10. Autenticação Baseada em *Passwords*

## 10.1 Conceitos e Fatores

- **Identificação**: quem o utilizador afirma ser (username). **Autenticação**: prova da identidade (password).
- **Fatores**: conhecimento (password, PIN), propriedade (token, telemóvel), característica (biometria).
- **Multi-fator**: combina fatores (ex.: password + código SMS).

## 10.2 Sistema de Autenticação

- **Não armazena passwords em claro:** armazena informação de validação ( $v = f(\text{password})$ ). Ex.: ( $v = H(\text{password})$ ).
- Função de validação: ( $g(v)(a) = (v = f(a))$ ).
- **Exemplo Linux:** `/etc/shadow` guarda `hash$salt$parâmetros`.

### 10.3 Ataques e Contramedidas

- **Ataque de dicionário tipo 2** (online): testa passwords na interface de autenticação.
- **Ataque de dicionário tipo 1** (offline): atacante possui ( $v$ ), testa offline.
- **Pré-computação (rainbow tables):** calcula ( $f(\text{password})$ ) para todo dicionário; com *salt* diferente por utilizador, o ataque torna-se inviável.
- **Contramedidas:**
  - *Salt* aleatório por utilizador.
  - Aumentar custo da função ( $f$ ) (ex.: iterações múltiplas, uso de funções pesadas como `bcrypt`, `Argon2`).
  - Políticas de bloqueio, atraso artificial (*backoff*), CAPTCHA.
  - Senhas fortes (comprimento, caracteres especiais).

### 10.4 Bcrypt e Argon2

- **Bcrypt:** custo parametrizável (iteraões), inclui *salt* automaticamente, saída no formato `$2b$custo$salt+hash`.
- **Argon2id** (vencedor do *Password Hashing Competition*, RFC 9106): complexidade de espaço (memória) para resistir a GPUs. Parâmetros recomendados OWASP: ex. ( $m=7168, \text{KiB}$ ,  $t=5$ ,  $p=1$ ).

---

## 11. Protocolo HTTP e Cookies

### 11.1 HTTP

- **Mensagens texto** (pedido/resposta). Métodos: `GET`, `POST`, `HEAD`, etc.
- **Stateless:** cada pedido é independente; a manutenção de estado é feita via *cookies* ou parâmetros.
- **Status codes:** `200 OK`, `302 Found` (redirecionamento), `401 Unauthorized`, `403 Forbidden`, `404 Not Found`.
- **HTTPS:** HTTP sobre TLS, porta 443.

### 11.2 Cookies

- Pequenos pares `nome=valor` que o servidor pede ao cliente para armazenar (*Set-Cookie*). O cliente reenvia em pedidos subsequentes (*Cookie*).
- **Propriedades:**
  - `Expires` / `Max-Age`: validade temporal.
  - `Domain` / `Path`: escopo de envio.
  - `Secure`: apenas em HTTPS.
  - `HttpOnly`: inacessível via JavaScript (`document.cookie`).
  - `SameSite`: restringe envio em pedidos *cross-site* (Strict, Lax).
- **Third-Party Cookies:** usados para *tracking* entre sites (ex.: anúncios). EU considera dados pessoais (GDPR).
- **Ataque de session hijacking:** se um atacante obtém o cookie de sessão, pode impersonar o utilizador.



---

## 12. Autenticadores em Contexto Web

### 12.1 Autenticação HTTP Basic/Digest

- **Basic:** cabeçalho `Authorization: Basic base64(utilizador:password)`. Inseguro sem TLS, expõe credenciais.
- **Digest:** envia hash de nonce+password para evitar exposição direta.
- Limitações: *stateless* obriga envio constante das credenciais.

### 12.2 Autenticadores com Cookies

- **Fase 1:** após validação positiva, servidor gera autenticador e envia como cookie.
- **Fase 2:** cliente inclui cookie em pedidos subsequentes; servidor valida autenticador.
- **Implementações:**
  1. **Identificador de sessão (stateful):** cookie contém ID aleatório; servidor guarda mapeamento ID→dados do utilizador. Necessita de base de sessões no servidor.
  2. **MAC (stateless):** autenticador contém dados do utilizador + MAC calculado com chave secreta do servidor. O servidor pode verificar a integridade sem armazenamento, mas precisa de lista de revogação para *logout*.
- **Propriedades de segurança:**
  - Cookie de autenticação deve ser `Secure` e `HttpOnly`.
  - `SameSite=Strict` para prevenir **CSRF**.
  - Validade temporal controlada pelo servidor (não confiar apenas no `Expires` do cookie).
  - Suporte a *logout*: invalidar sessão na base (stateful) ou adicionar token a uma lista de revogação.

### 12.3 Autenticação com Tokens JWT

- Adequado quando o serviço de autenticação está em domínio separado.
- **JWT** (*JSON Web Token*): formato padronizado (RFC 7519) com três partes codificadas em Base64: `header.payload.signature`.
  - **Header** ex.: `{"alg":"HS256","typ":"JWT"}`.
  - **Payload** (claims): `sub`, `name`, `iat`, `exp`, etc.
  - **Signature:** HMAC (HS256) ou assinatura digital (RS256).
- **Fluxo:**
  1. Cliente redirecionado para servidor de autenticação, que gera JWT após login.
  2. Servidor de autenticação redireciona de volta à aplicação incluindo JWT como parâmetro (ou fragmento).
  3. Aplicação verifica a assinatura (com chave partilhada ou pública) e extrai identidade.

---

## 13. OAuth 2.0

### 13.1 Objetivos e Papéis

- **Autorização condicionada:** um utilizador (*Resource Owner*) pode conceder a uma aplicação (*Client*) acesso limitado a recursos protegidos (*Resource Server*) sem partilhar as suas credenciais.
- **Authorization Server:** autentica o *Resource Owner* e emite *Access Tokens*.
- **Tipos de Client:** confidencial (pode guardar `client_secret` em segurança) vs. público.

## 13.2 Fluxos de Autorização

- **Authorization Code Grant** (recomendado):
  1. Client redireciona *user-agent* para **authorization endpoint** com parâmetros (**response\_type=code**, **client\_id**, **scope**, **state**, **redirect\_uri**).
  2. Resource Owner autentica e consente.
  3. Authorization Server redireciona para **redirect\_uri** com **code** e **state**.
  4. Client troca o **code** por um **access token** (e opcional **refresh token**) via *back channel* (**POST /token**, autenticando-se com **client\_id** e **client\_secret**).
- **Implicit Grant**: token devolvido diretamente no redirecionamento (fragmento). Menos seguro, obsoleto no OAuth 2.1.
- **Resource Owner Password Grant**: client recebe as credenciais do utilizador diretamente (apenas para clientes de elevada confiança).
- **Client Credentials Grant**: client usa as suas próprias credenciais para obter token (sem utilizador). Ex.: máquina-para-máquina.

## 13.3 Tokens e Scopes

- **Access Token**: credencial que o client apresenta ao *Resource Server*. Geralmente opaco, pode ser JWT. Tipo *Bearer* (posse = autorização).
- **Refresh Token**: permite obter novos *access tokens* sem interação do utilizador.
- **Scope**: *strings* que definem os limites do acesso (ex.: **read**, **write**). Customizáveis pelo serviço.

## 13.4 Front Channel vs. Back Channel

- **Front Channel**: interações através do *user-agent* (redirecionamentos). Informações sensíveis (credenciais do client, tokens de acesso) não devem passar por aqui.
- **Back Channel**: comunicação direta entre client e authorization server (protegida por TLS). Usada para trocar **code** por **token**.

---

# 14. OpenID Connect

## 14.1 Extensão do OAuth 2.0 para Autenticação

- **Papéis**: *End-User*, *Relying Party* (RP – a aplicação), *OpenID Provider* (OP – o fornecedor de identidade, combina Authorization Server e Resource Server).
- **Scope openid** indica pedido de autenticação.
- **ID Token**: JWT assinado pelo OP, contendo *claims* sobre o *End-User* (ex.: **sub**, **iss**, **aud**, **exp**, **iat**, **nonce**). A RP valida a assinatura e as *claims*.
- **UserInfo Endpoint**: recurso protegido por *access token* que devolve atributos adicionais do utilizador (JSON).

## 14.2 Fluxo *Authorization Code* no OpenID Connect

- Segue o mesmo fluxo OAuth 2.0, adicionando **scope=openid**.
- Na resposta do *token endpoint* o OP devolve simultaneamente **access\_token** e **id\_token**.
- A RP pode, de seguida, obter **UserInfo** com o **access\_token**.

## 14.3 Múltiplos Scopes

- O mesmo fluxo pode combinar autenticação (`openid`) e autorização para APIs (ex.: `photos.read`), resultando num *access token* válido para ambas as finalidades.
- 

## 15. Ataques e Contramedidas no OAuth 2.0 / OpenID Connect

### 15.1 Modelo de Atacante

- Acesso total à rede entre client, authorization server e resource server (pode observar e modificar tráfego).
- Recursos ilimitados; pode criar servidores maliciosos.
- Partes podem estar em conluio.

### 15.2 Ameaças Comuns

- **Comprometimento do `client_secret`** (ex.: hardcoded no código fonte). Contramedida: guardar em ficheiro de configuração, registar segredos fortes, revogar.
- **Exposição de tokens**: usar TLS, limitar escopo e tempo de vida.
- **CSRF via `authorization code`**: atacante inicia fluxo, obtém código, força vítima a usá-lo. O `state` parameter (verificado pelo client) previne.
- **Phishing**: client malicioso redireciona para falso authorization endpoint. Educação do utilizador, uso de indicadores visuais.
- **Ataque de repetição** de tokens: *nonce* no `id_token`, *state* e *redirect\_uri* verificados.
- **Ataque de força bruta** contra o *token endpoint*: limitação de tentativas, credenciais fortes.

### 15.3 Proteções Indispensáveis

- **TLS** em todas as comunicações.
  - **Parâmetro `state`** em fluxos com redirecionamento.
  - Validação rigorosa da `redirect_uri` (contra *open redirector*).
  - Armazenamento seguro de `client_secret` e *refresh tokens*.
- 

## 16. Modelos de Controlo de Acesso e RBAC

### 16.1 Conceitos Gerais

- **Controlo de Acesso**: políticas + mecanismos para restringir e auditar acessos. Inclui identificação, autenticação, autorização.
- Arquitetura do **Monitor de Referências**:
  - **PEP** (*Policy Enforcement Point*): intercepta pedidos e aplica a decisão.
  - **PDP** (*Policy Decision Point*): avalia políticas e decide.

### 16.2 Modelo de Matriz de Acessos

- Linhas: sujeitos; colunas: objetos; células: permissões.
- **Implementações**:
  - **ACL** (no objeto): lista de quem pode aceder e operações.

- **Capabilities** (no sujeito): lista de recursos que pode aceder.
- **Tabelas de autorização**: triplos (sujeito, permissão, objeto).
- **Grupos**: vários utilizadores podem partilhar permissões. Ex.: Linux (**owner**, **group**, **other**).

### 16.3 Exemplo Windows

- **SID** (*Security Identifier*).
- **DAACL**: lista de ACEs. Cada ACE especifica SID, ações, resultado (Allow/Deny). Processamento sequencial; na primeira ACE correspondente, a decisão é aplicada.
- **SACL**: lista de auditoria.

### 16.4 Role-Based Access Control (RBAC)

- Motivação: permissões atreladas a papéis (*roles*), não diretamente a utilizadores. Adequado a organizações.
- **Níveis** (Sandhu et al.):
  - **RBAC<sub>0</sub>**: base.
    - Conjuntos: (U) (utilizadores), (R) (papéis), (P) (permissões), (S) (sessões).
    - Relações:  $UA \subseteq U \times R$  (atribuição de papéis),  $PA \subseteq R \times P$  (atribuição de permissões).
    - Função:  $(roles(s_i) \subseteq \{ r \mid (users(s_i), r) \in UA \})$  – sessão ativa um subconjunto dos papéis do utilizador (*princípio do privilégio mínimo*).
  - **RBAC<sub>1</sub>**: adiciona hierarquia de papéis ((RH)).
    - $(r_x \subseteq r_y)$ : papel sénior herda permissões do júnior.
    - Herança transitiva.
    - **Private roles**: papéis especiais usados para evitar herança indesejada.
  - **RBAC<sub>2</sub>**: adiciona restrições sobre RBAC<sub>0</sub>.
    - Cardinalidade, pré-requisitos, exclusão mútua (estática/dinâmica) – suportam **separação de deveres**.
  - **RBAC<sub>3</sub>**: combina RBAC<sub>1</sub> + RBAC<sub>2</sub>.
- **Limitações**: explosão de papéis (muitas condições especiais), complexidade de gestão.
- **ABAC** (*Attribute-Based Access Control*): autorização baseada em atributos do sujeito, objeto e ambiente (ex.: hora do dia, localização). Maior flexibilidade.

### 16.5 Legislação e RGPD

- Necessidade de perfis de privilégio mínimo.
- Auditoria de tentativas de acesso não autorizado e ações sobre dados pessoais.

---

## 17. Biblioteca Casbin

### 17.1 Conceitos Básicos

- Implementa o **PDP** (Policy Decision Point). A aplicação deve integrar o PEP.
- **Ficheiros de configuração**:
  - **model.conf**: define o modelo (ACL, RBAC, ABAC) – secções **[request\_definition]**, **[policy\_definition]**, **[policy\_effect]**, **[matchers]**, **[role\_definition]**.
  - **policy.conf**: políticas concretas (atribuições de papéis, permissões).

- **Sintaxe RBAC:**
  - `g, alice, admin` → atribuição de papel.
  - `g, admin, coordinator` → hierarquia (admin sénior de coordinator).
  - `p, coordinator, data1, write` → permissão.
- **Matchers:** `g(r.sub, p.sub) && r.obj == p.obj && r.act == p.act` – avalia se pedido casa com política considerando herança de papéis.
- **Enforcer** em Node.js: `const enforcer = await newEnforcer('model.conf', 'policy.conf');` e `enforcer.enforce(sub, obj, act)`.

## 17.2 Integração como Middleware

- Exemplo: extrair utilizador da sessão, rota e método HTTP; chamar `enforce`; se `false`, redirecionar ou negar.

## 17.3 Editor Casbin

- Ferramenta web para testar modelos e políticas (<https://casbin.org/editor/>).

---

## Termos-Chave

**Confidencialidade, Integridade, Disponibilidade, Autenticidade, Não-repúdio, Criptografia simétrica/assimétrica, Cifra de bloco, Modo de operação (ECB, CBC, CTR, GCM), Padding PKCS#5, IV, MAC, Função de hash criptográfico, HMAC, RSA, Certificado digital X.509, CA, Cadeia de certificação, Trust anchor, TLS, Handshake, Record Protocol, Cipher Suite, Perfect Forward Secrecy, Diffie-Hellman, JCA, Cipher, Signature, KeyStore, HTTP, Cookie, Session, JWT, OAuth 2.0, Authorization Code Grant, Access token, Refresh token, Scope, OpenID Connect, ID token, CSRF, RBAC (RBAC0/1/2/3), PEP, PDP, Casbin, Modelo de controlo de acessos, ABAC.**